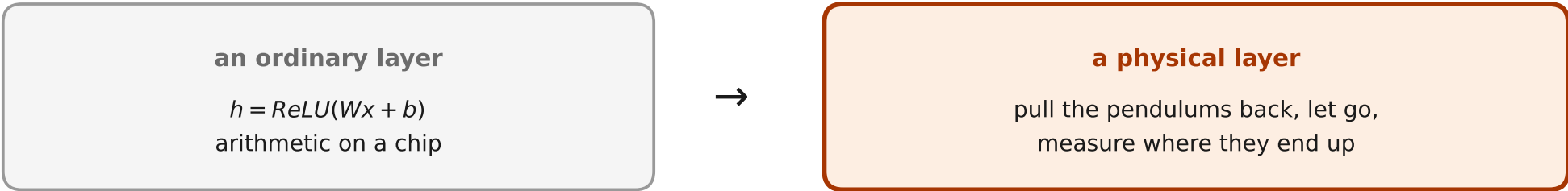


Physics-Aware Training — the whole thing on one page

come back here whenever a detail page stops making sense

THE IDEA



same job — mix the inputs, bend them, produce outputs. The weights are the spring stiffnesses.

THE PROBLEM



backprop needs the derivative of every layer. A lump of metal has no code, so it has no derivative.
and you cannot measure your way out: a measurement is a value, not a slope. One poke per knob = 2.6 million runs.

THE FIX



run it, measure it differentiated at the plate's measured values

the simulation is wrong, so the gradient is approximate — but reality re-anchors the state at every layer,
so the error never compounds. The weights stay on your computer throughout and change only in *optimizer.step()*.

THE RESULT

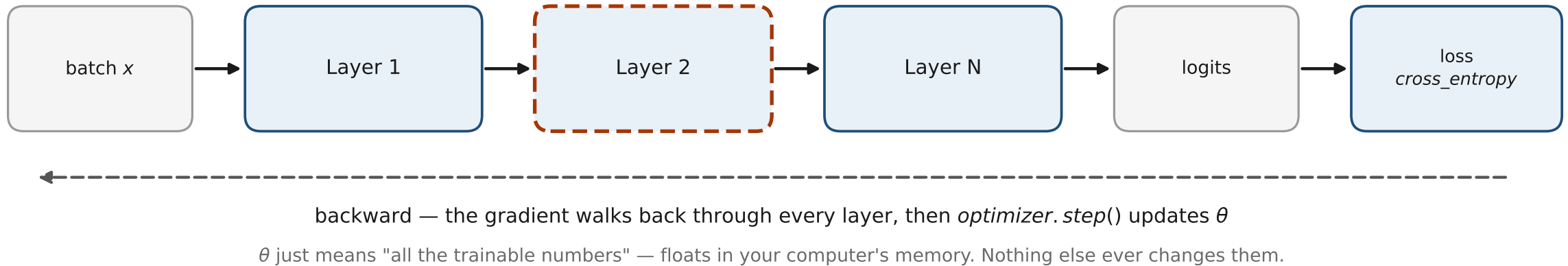
PAT **87.80%** trained on the simulation only **79.25%**

both measured on the real device. The simulation-trained one reports 87.55% to itself — it never tells you anything is wrong.

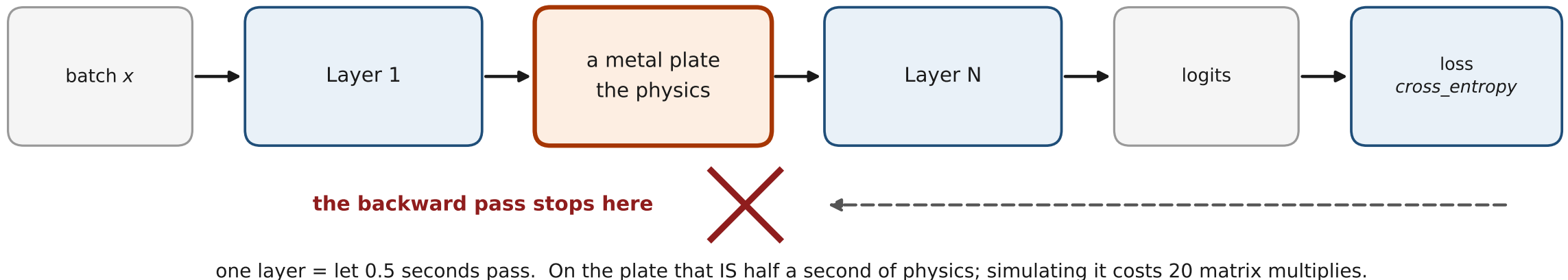
Step 1 The loop you know, and the one box that breaks it

generic picture: a transformer, or any network. This repo's own network is different — see Steps 6 and 7.

AS YOU KNOW IT



WITH A PHYSICAL LAYER



UNCHANGED

- ✓ the data loading, the other layers, the logits
- ✓ the loss — still `F.cross_entropy( $\hat{y}$ , label)`
- ✓ the optimizer — still Adam
- ✓ θ — still ordinary floats on your computer

BROKEN

- ✗ the plate has no source code
- ✗ autograd recorded nothing on the way through
- ✗ there is no derivative to walk back
- ✗ so this layer cannot be trained

That single broken arrow is the entire problem. Everything that follows is about getting a gradient past it.

Step 2 **A measurement is not a gradient**

"but we ran the real plate and measured the output — why isn't that enough?"

what the device gives you

a VALUE

"with these knobs and this input,
this came out" — one point

what training needs

a SLOPE

"if I nudged knob #4,192 a little,
how would the loss change?"

you cannot read a slope off a single point — a rate of change needs at least two measurements

the same is true of your transformer: knowing the logits tells you nothing about $\partial \text{loss} / \partial W$ until you run backward

You could get the slopes by measurement alone...

measure the loss → nudge one knob → run the plate again → the difference IS that knob's derivative, exactly

then repeat for the next knob. 2,603,811 hardware runs, per gradient step. Honest, and hopeless at any real scale.

...so why is backprop cheap? Because it knows the STRUCTURE.

it knows every operation that produced the output, so it works out all 2.6 million slopes analytically, in one pass.

measurement knows no structure. It sees a black box, so it must discover each slope separately, one poke at a time.

the device

truth ✓ structure ✗

the simulation

truth ✗ structure ✓

PAT takes from each what it actually has: the device supplies the values, the simulation supplies the slopes.

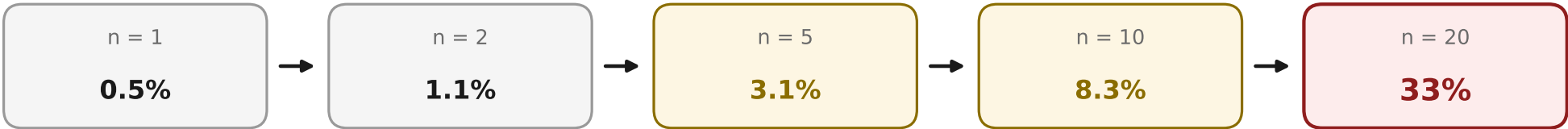
Fog on a hillside: the altimeter says exactly where you are, the map says which way is down. Read the map at the altimeter's position.

Step 3

The four ways out

two dead ends, one impossible, and one that works.

	touches hardware during training	can the physics layer learn?	cost per training step	result
A freeze it leave the knobs where they are; train only the digital parts around the plate	no	X no	cheap	wastes the device
B simulate both ways train entirely in software, then set the real dials to the finished numbers	no	✓ for the wrong device	cheap	79.25%
C PAT run the real plate; let the simulator differentiate AT the values it measured	yes, every batch	✓ yes	one round trip per layer	87.80%
D measure it directly Why B loses: suppose the simulation is 0.5% off. After n layers the gap is — nudge a knob, re-run the plate, read off the difference — exact, and unaffordable	yes, millions of times	✓ yes	2.6M runs per step	impossible



errors compound, they do not add — every layer works on the previous layer's already-wrong answer.

and B never warns you: on its own simulator it reports 87.55%, a quarter-point above PAT's real score of 87.80%.

Step 4

What the simulator is

a piece of code you write that predicts the device. Nothing more exotic than that.

$f_{model}(x, Q, e) \rightarrow$ a prediction of what the device will do

same inputs as the real device: the data, the coupling settings, the drives

in this repo that is $\ddot{q} = -\sin q + Q \sin q + e$, solved with RK4, 5 steps — your physics homework, in code

it exists only to be differentiated

vjp does evaluate it — you cannot differentiate a function without running it — but its prediction is discarded. Only the gradient leaves.

where it comes from

- write down the physics you believe governs the device
- measure its constants as best you can — masses, stiffnesses, damping
- code it in torch, so autograd can differentiate it
- or: poke the device with many inputs, record the outputs, fit a small net to that

why it is always wrong

- your measured constants are a bit off
 - the device has effects you never modelled — stray couplings, drift, manufacturing quirks
 - your equations were an idealisation anyway
- you can shrink the gap. You cannot close it.**

One confusing thing about this repo: there is no hardware, so the device is faked too

`f_model = make_ode_map(ode_model, Nt, dt)`

← *the simulator*

`f_exp_small = make_ode_map(ode_exp_small, Nt, dt)`

← *code pretending to be the metal plate*

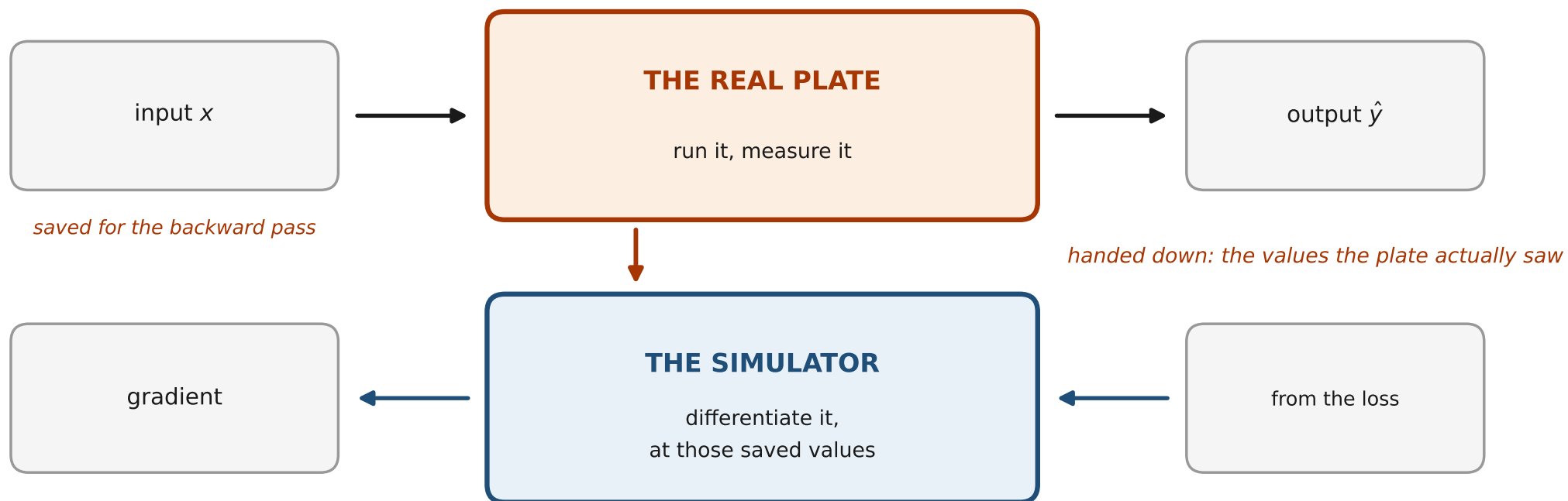
f_{exp} is the same equation, deliberately spoiled: a 10% error in the nonlinearity (η) plus stray couplings Q_{noise} the simulator knows nothing about.

In the paper it was a speaker-driven metal plate, a nonlinear optical crystal, an analog circuit — real objects on a bench.

PAT: the device forward, the simulator backward

one substitution, thirty lines of code. Everything else in the training loop is untouched.

FORWARD



pat.py — the whole method

```
class func(torch.autograd.Function):
```

```
    def forward(ctx, *args):
```

```
        ctx.save_for_backward(*args)
```

```
        return f_forward(*args)
```

← save this layer's INPUTS — the real ones

← the device runs here, with no graph recorded

```
    def backward(ctx, grad_output):
```

```
        args = ctx.saved_tensors
```

```
        torch.set_grad_enabled(True)
```

```
        y = vjp(f_backward, args, v=grad_output)
```

```
        return y[1]
```

← the values the device actually saw

← grad mode switches on only now

← $y[0]$ = the model's guess — discarded

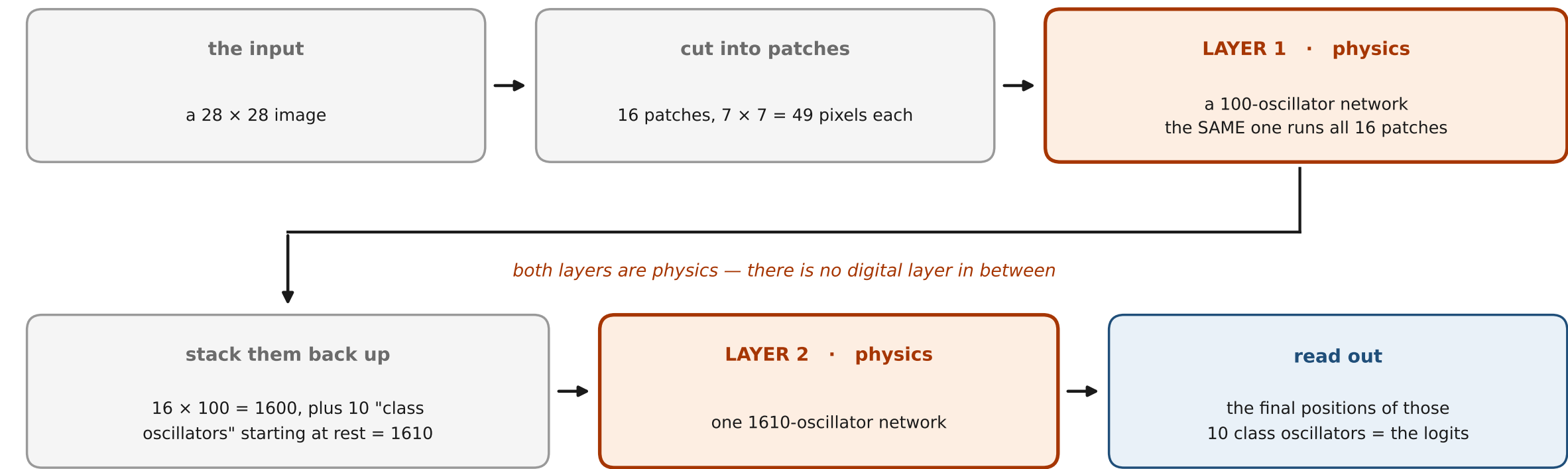
← $y[1]$ = the gradient — the only thing used

the gradient is approximate — it comes from a model that is wrong. It only has to point roughly downhill.

Darts: you watch where each dart actually lands, while your sense of how to adjust your arm stays imperfect. Corrected against a real outcome every throw, the flaws never pile up.

Step 6 **This repo, specifically**

two physical layers — not a generic stack — and 2.6 million numbers that never leave your computer.



θ — **every trainable number in the model**

<code>fc_small.weight</code>	100 × 100	10,000	spring stiffness between every pair, layer 1
<code>fc_small.bias</code>	100	100	the push applied to each oscillator, layer 1
<code>fc_large.weight</code>	1610 × 1610	2,592,100	spring stiffnesses, layer 2
<code>fc_large.bias</code>	1610	1,610	the pushes, layer 2
<code>output_fac</code>	one number	1	a single output scale
TOTAL		2,603,811	ordinary floats on your computer — never inside the plate

They change in exactly one place, and it is the same place as in any torch model: `optimizer.step()`.

both layers are physics. f_{model} never appears in the forward pass — it is only ever called on the way back.

The wiring — two lines, and this is the whole mapping

```
f_pat_small = make_pat_func(f_exp_small, f_model)
f_pat_large = make_pat_func(f_exp_large, f_model)
```

1st argument = the FORWARD pass = the device

2nd argument = the BACKWARD pass = the simulator

Where those get called — *PNN.forward*

```
def forward(self, x):
    x = self.rearrange(x)
    ...
    x = f_pat_small(x, *self.fc_small.parameters())
    ...
    x = f_pat_large(x, *self.fc_large.parameters())
    return self.output_fac * x[:, -10:, 0]
```

← 16 patches of 49 pixels

← LAYER 1 · physics

← LAYER 2 · physics

← the 10 class oscillators = the logits

So, per batch:

	the FORWARD pass runs	the BACKWARD pass runs
layer 1 · 100 oscillators	f_{exp_small} the device	f_{model} at (x_0, C_1, e_1)
layer 2 · 1610 oscillators	f_{exp_large} the device	f_{model} at (x_1, C_2, e_2)
the loss	$F.cross_entropy$	ordinary autograd — nothing special

f_{model} appears zero times in *forward()*. f_{exp} appears zero times in *backward()*.

Neither is a layer of the network — they are two directions through the same one.